Gabriel BELTZER, Owen BONSERGENT & Titouan HINSCHBERGER - Master ISA Tours

Web of Things - Project 3 Report

Objectif

Ce projet avait pour objectif de concevoir une architecture REST permettant de simuler un scénario relevant du *Web of Things* (WoT). À partir d'un squelette de projet fourni, basé sur Spring Boot et Maven, il s'agissait de compléter l'architecture existante, de développer de nouveaux objets connectés (*Things*) et de mettre en place leur communication, dans le respect des conventions des API REST.

Scénario de base et extension réalisée

Le scénario de base reposait sur trois objets connectés : une lampe, un thermostat et un capteur de mouvement. La détection d'un mouvement devait entraîner l'allumage de la lampe et l'ajustement de la température cible du thermostat afin d'augmenter la température ambiante si nécessaire. Des règles complémentaires régissaient notamment l'extinction de la lampe et le changement de mode du thermostat en fonction du temps écoulé depuis la dernière détection de mouvement ou de la température mesurée. Leur implémentation sera détaillée dans une section ultérieure.

Un compteur-disjoncteur a ensuite été ajouté afin d'enrichir le scénario et de multiplier les interactions entre les objets. Ce composant calcule les consommations électriques instantanée et cumulée du système, en tenant compte de la consommation des trois autres objets, de leur état de fonctionnement et des différents coûts associés. Il permet également de couper ou de rétablir l'alimentation électrique de l'ensemble du circuit, afin d'arrêter ou de remettre en marche tous les appareils au moyen d'une seule action.

Architecture

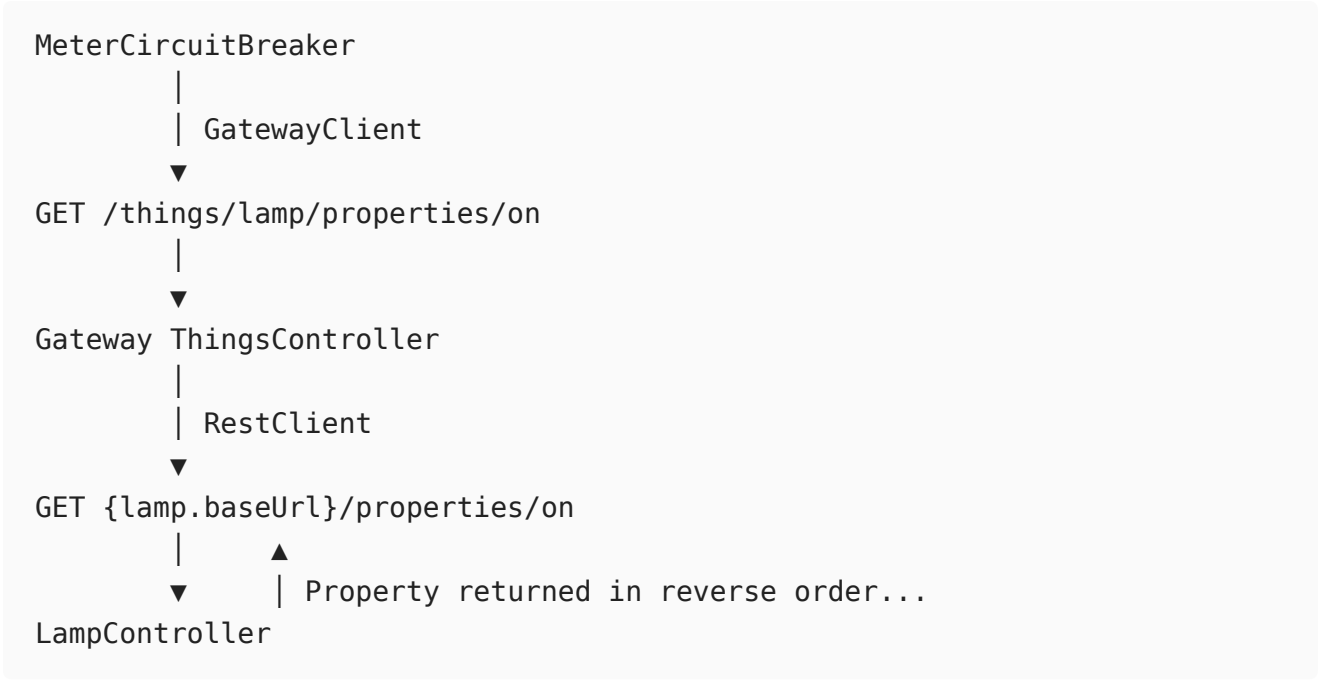
Le réseau d'objets connectés développé repose sur une architecture REST implémentée avec Spring Boot. Chaque composant s'exécute dans un processus distinct et expose une API HTTP sur son propre port. Un module a ainsi été créé pour chaque objet, ainsi que pour la passerelle (*gateway*), accessible sur le port 8080.

La passerelle occupe une place centrale dans l'architecture. Elle assure le rôle de **proxy**, en recevant les requêtes HTTP destinées aux objets et en les transmettant aux composants concernés. Elle prend également en charge l'implémentation des règles métier déclenchées par les événements émis par les objets, tels qu'un mouvement détecté, l'atteinte de la température cible ou le changement d'état du disjoncteur.

Chaque module correspondant à un objet est organisé autour des composants suivants :

- **Contrôleur** : expose les différents points d'accès (*endpoints*) de l'API et traite les requêtes HTTP, notamment les méthodes GET, POST et PUT. Il peut également intégrer la logique propre à l'objet, présentée plus loin.
- **Descripteur** : décrit le modèle de l'objet en définissant ses propriétés, ses actions, ses événements et ses liens.
- **Client de communication avec la passerelle** (*GatewayClient*) : assure les échanges entre l'objet et la passerelle, notamment pour son enregistrement, l'accès aux autres objets et la transmission d'événements.
- **Application** : contient le point d'entrée du module, permettant de démarrer l'application Spring Boot correspondante.

L'exemple suivant illustre une requête effectuée par un objet vers un autre, en passant par la passerelle. Le compteur-disjoncteur doit notamment connaître l'état de la propriété *on* de la lampe afin de déterminer si sa consommation électrique doit être prise en compte dans le calcul de la consommation du système.



Cet exemple illustre l'intérêt du proxy : les objets n'ont pas besoin de connaître les URL des autres objets. Ils identifient les composants avec lesquels ils souhaitent interagir à l'aide de leur identifiant (*ID*) et passent systématiquement par la passerelle pour communiquer avec eux.

Logique d'exécution des règles

Les règles implémentées permettent de reproduire le comportement attendu dans le scénario simulé. Elles sont principalement déclenchées par les événements émis par les objets, tels que la détection d'un mouvement ou l'atteinte de la température cible. Leur

exécution est donc gérée par le contrôleur d'événements de la passerelle (*gateway/EventsController*).

Comme le montre l'extrait de code suivant, la réception d'un événement **motion** entraîne l'allumage de la lampe (propriété *on* définie à *true*) et le passage du thermostat en mode *heat*, avec une température cible fixée à 19 °C. Ces actions correspondent aux règles **R1** et **R2** du scénario.

```
@PostMapping
public ResponseEntity<Void> receive(@RequestBody ThingEvent event) {
  // ... There is code before this
  // R1 and R2
  if("motion".equals(event.type())){
    if(circuit_live) {
      // ... There is code before this
      things.writeProperty("lamp", "on", "{\"value\":true}");
      lampOnByR1 = true;
      if(temp < 19.0){
        things.writeProperty("thermostat", "target", "{\"value\":19.0}");
                        things.writeProperty("thermostat",   "mode",   "
{\"value\":\"heat\"}");
      }
    }
  }
  // R2 - end
  if("targetReached".equals(event.type())){
    things.writeProperty("thermostat", "mode", "{\"value\":\"off\"}");
  }
  return ResponseEntity.accepted().build();
}
```

Les conditions de fin des règles **R1** et **R2**, ainsi que le déclenchement de la règle **R3**, qui prévoit le passage du thermostat en mode *eco*, dépendent également du temps écoulé depuis certains événements. Leur prise en charge repose sur des tâches périodiques, mises en place à l'aide de l'annotation *@Scheduled* de Spring. L'extrait de code suivant illustre ce mécanisme de vérification régulière des conditions temporelles et d'exécution des actions associées.

```
@Scheduled(fixedDelay = 1000)
public void checkTimers() {
  if (!state.isCircuitLive()) return;
  try {
    Instant lastMotion = readLastMotion();
    String mode = readThermoMode();
    // au bout de T1, on eteint la lampe si pas de motion depuis
    // (sauf si elle a été allumée à la main)
    if (lastMotion != null
```

```
            && state.isLampOnByR1()
            && !state.isLampManual()
            && Duration.between(lastMotion, Instant.now()).toMillis() >= t1Ms)
{
      things.writeProperty("lamp", "on", "{\"value\":false}");
      state.setLampOnByR1(false);
    }
    // au bout de T3, on passe en mode eco et target 17 si pas de motion
depuis
    // (sauf si le mode a été choisi à la main)
    if (lastMotion != null
        && mode != null
        && !mode.equals("eco")
        && !state.isThermostatManual()
        && Duration.between(lastMotion, Instant.now()).toMillis() >= t3Ms)
{
      things.writeProperty("thermostat", "mode", "{\"value\":\"eco\"}");
      things.writeProperty("thermostat", "target", "{\"value\":17.0}");
    }
  } catch (Exception e) {
    log.warn("Rule check failed: {}", e.getMessage());
  }
}
```

En revanche, la logique interne propre à chaque objet, lorsqu'elle ne dépend pas directement des événements échangés entre les composants, est implémentée dans le contrôleur de l'objet concerné. C'est notamment le cas du thermostat, dont la température évolue progressivement en fonction de son mode de fonctionnement et de la température cible définie. Lorsque la température mesurée atteint cette cible, le thermostat émet un événement pour signaler ce changement aux autres composants du système.

```
@Scheduled(fixedDelay = 1000)
public void addaptHeat() {
    String mode = (String) state.get("mode");
    double temp = ((Number) state.get("temperature")).doubleValue();
    double target = ((Number) state.get("target")).doubleValue();
    boolean reached = false;
    if("off".equals(mode)) {
        if(temp == min_temp) return;
         temp = Math.max(min_temp, Math.round((temp+negative_step)*100.0)
/ 100.0);
    }
    else {
         if (temp < target) temp = Math.round((temp+step)*100.0) / 100.0;
         if (temp >= target) reached = true;
    }
    set("temperature", temp);
     if(reached)  gateway.emit("targetReached",  Map.of("target",  target,
```

```
"temperature", temp));
}
```

Mécanismes de sécurité

Nous avons implémenté un Token Bearer vérifié dans un filtre HTTP (*gateway/TokenFilter*) exécuté avant les contrôleurs. Il s'applique à toutes les routes `/things/**` et `/events/**`, alors que les fichiers du tableau de bord (*static/index.html*) restent publics pour pouvoir afficher la page de connexion. Les objets s'authentifient eux aussi avec ce jeton quand ils s'enregistrent et envoient leurs évènements à la passerelle (`gateway.token` dans leur *application.yml*).

Deux rôles ont été définis, configuré dans le *application.yml* de la passerelle:

- **operator** (`gateway.token`) qui a tous les droits : lecture, écriture des propriétés, actions, enregistrement et suppression d'objets
- **viewer** (`gateway.viewer-token`) qui est en lecture seule et ne peut utiliser que les méthodes GET, HEAD et OPTIONS

```
gateway:
 # Authorization: Bearer <token> on /things/\*\* and /events/\*\*
 token: operator-secret
 # read-only role: GET allowed, PUT/POST/DELETE -> 403
 viewer-token: viewer-secret
```

Le filtre distingue trois cas, ce qui permet de séparer une authentification manquante (**401**) d'un droit insuffisant (**403**) :

```
if (matches(token, expectedToken)) {
 chain.doFilter(request, response); // operator : tout est permis
} else if (matches(token, viewerToken)) {
 if (READ\_METHODS.contains(request.getMethod())) {
   chain.doFilter(request, response); // viewer : lecture seulement
 } else {
   reject(response, 403, "the viewer role is read-only");
 }
} else {
 response.setHeader(HttpHeaders.WWW\_AUTHENTICATE, "Bearer");
 reject(response, 401, "missing or invalid token"); // absent ou incorrect
}
```

Quelques détails :

- le token est lu dans l'en-tête `Authorization: Bearer <token>` , sauf pour `/events/stream` où il peut aussi être passé en paramètre (`?token=...`)

- un token vide dans la configuration n'est jamais accepté : si `viewer-token` n'est pas défini, le rôle viewer est simplement désactivé et ses requêtes reçoivent un 401

Les tests dans *check-routes.sh* vérifie ces comportements : sans token ou avec un mauvais token on obtient 401, avec le token viewer la lecture de `/things` renvoie 200 alors que `PUT` d'une propriété et `POST` d'une action renvoient 403.

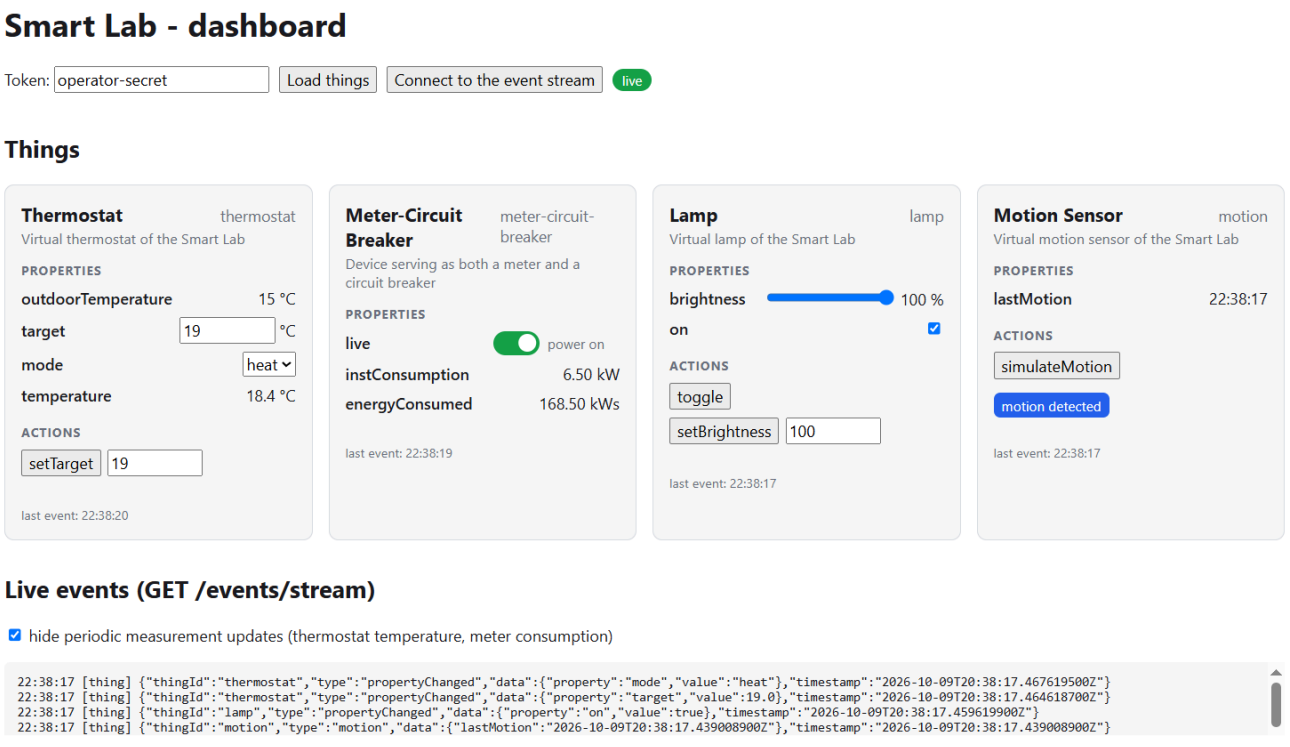
```
VIEWER=viewer-secret ./check-routes.sh
```

Interface graphique

Afin de faciliter la simulation et les tests du système, une interface web graphique a été développée sous la forme d'un tableau de bord (*dashboard*).

Le front-end initial de la passerelle, situé dans `static/index.html`, a été enrichi par l'ajout de panneaux (*panels*) dédiés à chaque objet connecté. Ceux-ci permettent de consulter en temps réel les propriétés des objets et d'exécuter directement leurs actions depuis l'interface, sans avoir à formuler manuellement des requêtes HTTP, que ce soit en ligne de commande ou via le navigateur.

Enfin, l'interface permet également de visualiser le flux des événements reçus par la passerelle, facilitant ainsi le suivi des interactions entre les objets et l'observation du comportement global du système.



Améliorations possibles

L'architecture développée présente une bonne modularité et peut être étendue relativement facilement, notamment par l'ajout de nouveaux objets connectés ou la modification du comportement des objets existants. L'intégration du compteur-disjoncteur illustre cette capacité d'évolution, puisqu'elle a permis d'enrichir le scénario initial sans remettre en cause l'organisation générale du système.

L'axe d'amélioration principal consiste donc à enrichir l'environnement connecté afin de rendre la simulation plus complète et réaliste. Par exemple, le compteur-disjoncteur pourrait proposer des actions permettant de couper ou de rétablir individuellement l'alimentation de chaque appareil, à la manière des différents circuits d'un tableau électrique domestique. Il serait également possible d'ajouter de nouveaux objets, tels que des capteurs de luminosité ou des volets roulants automatisés, afin de diversifier les interactions et de mettre en œuvre des scénarios domotiques plus élaborés.

Une autre piste d'amélioration concernerait la gestion des règles d'automatisation. Celles-ci pourraient être rendues configurables, par exemple en permettant de modifier les seuils de température, les délais d'inactivité ou les conditions de déclenchement sans intervenir directement dans le code de la passerelle. Une telle évolution faciliterait l'adaptation du système à différents scénarios d'utilisation.

L'interface graphique pourrait également être enrichie par l'ajout de diagrammes afin de suivre l'évolution de la consommation dans le temps, par exemple.

Des tests automatisés couvrant les interactions entre les composants contribueraient également à vérifier le bon fonctionnement du système lors de l'ajout de nouvelles fonctionnalités.

Ces différentes évolutions permettraient ainsi de poursuivre l'enrichissement du scénario tout en conservant les principaux atouts de l'architecture : sa modularité, son extensibilité et la séparation des responsabilités entre les composants.

Récapitulatif du travail effectué

	Gabriel	Owen	Titouan
Things creation and implementation	✔	✔	✔
All rules and logic of objects		✔	✔
Proxy of the gateway	✔		
Dashboard	✔	✔	✔
Security with token	✔		
Liveness checking of things		✔	
OpenAPI specification (gateway.yaml)	✔		
Report	✔		✔

	Gabriel	Owen	Titouan
Demo video		✔	
Installation guide (README)			✔